

Cutter++

Interactive Modern C++ for Binary Analysis

Cutter++ is a plugin for Cutter that brings an interactive C++ REPL directly into your reverse engineering workflow.

Instead of writing Python scripts or compiling native plugins, you can write and execute modern C++ (C++20/C++23) snippets against the currently opened binary in real time.

Powered by **ICPP**, Cutter++ turns C++ into a scripting language for reverse engineering.

Why?

Reverse engineers frequently need to automate repetitive analysis tasks:

- Rename functions
- Traverse control-flow graphs
- Analyze cross references
- Search for patterns
- Patch binaries
- Build custom analysis tools

Today there are generally two options:

- **Python scripting**
 - Fast iteration
 - Dynamic
 - Limited access to existing C++ libraries
 - No templates or compile-time programming
- **Native C++ plugins**
 - Maximum performance
 - Full language support
 - Slow development cycle
 - Requires compilation and restarting Cutter

Cutter++ fills the gap by combining the convenience of scripting with the power of modern C++.

Features

- Interactive C++ REPL
- Execute modern C++20/C++23 code instantly
- Direct access to Cutter and Rizin APIs

- Native STL support
 - C++ Ranges support
 - Reuse existing C++ libraries
 - Persistent session state
 - Syntax highlighting
 - Command history
 - Script execution
 - Header auto-completion (planned)
 - Clang diagnostics (planned)
-

Example

```
for (auto &f : functions()) {  
    std::println("{} : {}", f.name(), f.size());  
}
```

Rename every small function:

```
for (auto &f : functions()) {  
    if (f.size() < 16)  
        rename(f, "tiny_" + f.name());  
}
```

Export a CFG:

```
auto cfg = build_cfg(currentFunction());  
  
cfg.export_graphviz("cfg.dot");
```

Hash the current selection:

```
auto bytes = currentSelection().bytes();  
  
std::println("{} ", sha256(bytes));
```

Find suspicious functions:

```
auto suspicious =  
    functions()  
    | std::views::filter(is_obfuscated)  
    | std::views::transform(&Function::name);
```

```
for (auto &name : suspicious)
    std::println("{} ", name);
```

API

Cutter++ exposes a modern C++ interface over Cutter and Rizin.

Examples include:

```
currentBinary()

currentFunction()

currentAddress()

functions()

imports()

exports()

strings()

sections()

segments()

comments()

xrefs()

graph()

disassemble()

decompile()

patch()

undo()

redo()
```

The API is designed to feel like standard C++, not a command shell.

Interactive Analysis

Instead of writing:

```
af1j
```

and parsing JSON,

simply write:

```
for (auto &f : functions())  
    std::println("{} ", f.name());
```

Instead of launching external scripts,

experiment interactively inside Cutter.

Powered by ICPP

Cutter++ uses **ICPP** as its execution engine.

ICPP allows C++ to be executed interactively with minimal startup overhead, making it practical to use C++ as an embedded scripting language.

This enables reverse engineers to use:

- Templates
- Concepts
- STL
- Ranges
- Existing C++ libraries

without leaving Cutter.

Use Cases

Malware Analysis

- Search for encryption routines
- Detect obfuscation
- Batch rename functions
- Automate IOC extraction

Firmware Analysis

- Traverse memory maps
- Analyze interrupt vectors
- Extract embedded resources

CTF

- Solve repetitive reversing challenges
- Rapid binary exploration
- Prototype exploit helpers

Compiler Research

- Inspect generated binaries
 - Analyze optimization passes
 - Compare compiler outputs
-

Roadmap

v0.1

- Interactive REPL
- Execute C++ snippets
- Basic Cutter API wrappers

v0.2

- Script files
- Session persistence
- Syntax highlighting

v0.3

- Auto-completion
- Clang diagnostics
- Better object browser

v0.4

- Debugger integration
- Memory inspection
- Live register access

v1.0

- Stable scripting API

- Plugin SDK
 - Documentation
 - Examples
-

Safety

Cutter++ executes native C++ code inside the Cutter process.

Like native plugins, executed code has the same permissions as the running application.

Only run trusted code.

Who is this for?

- Reverse engineers
- Malware analysts
- Security researchers
- CTF players
- Compiler developers
- Firmware researchers
- Binary instrumentation developers

If you already think in C++, Cutter++ lets you stay in C++ while exploring binaries.

Vision

Python made scripting popular.

Cutter++ brings the expressiveness, performance, and ecosystem of modern C++ to interactive reverse engineering.

The goal is not to replace Python.

The goal is to make **modern C++ a first-class language for binary analysis**.